

Relatório 12 - Prática: Pipelines de Dados I - Airflow (II)

Matheus Beuren Carvalho

1. Introdução

A tarefa atribuída pelo card, pediu a visualização de algumas seções do curso de airflow, onde por sua vez, é uma ferramenta muito útil e versátil. Ao longo do relatório, foi descrito os principais conceitos observados durante as aulas e principalmente o projeto que o curso faz durante uma de suas seções.

2. Desenvolvimento

O airflow é uma ferramenta de execução programada, sua utilização se baseia em coleta de dados e treinos de modelos, porém o airflow só é responsável por executar as tarefas e não realmente treinar um modelo. Tudo é separado por tarefas, e essas tarefas executam algum script ou algo que precisa ser programado para ser executado.

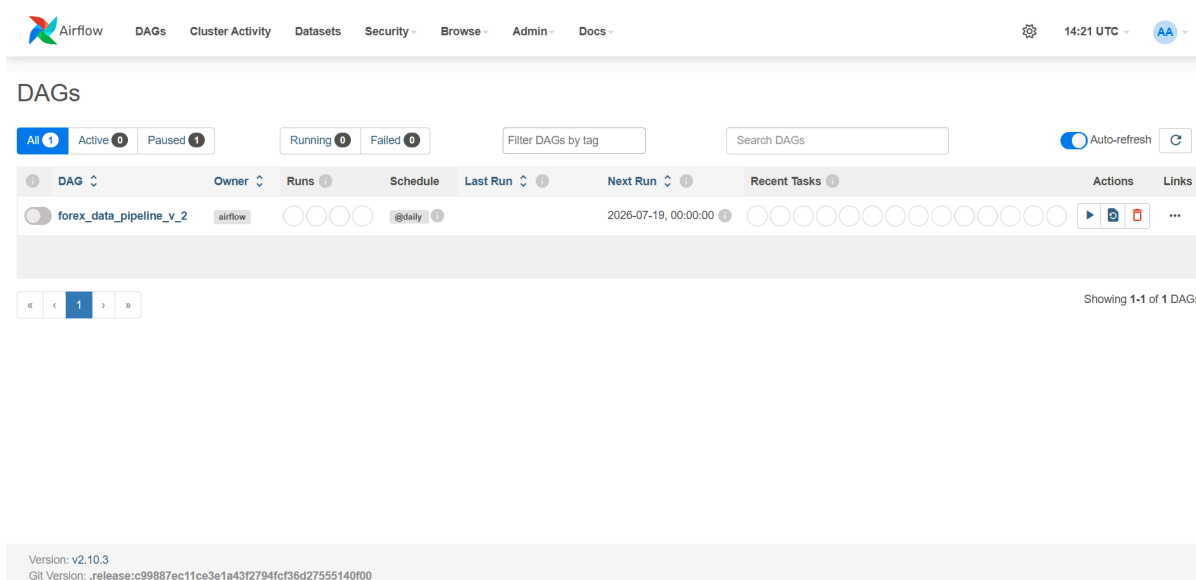
Os componentes principais são o scheduler, web server, metadata, executor e worker, eles são responsáveis por fazer o airflow funcionar. A seguir a (Tabela 1) consiste nos principais conceitos do airflow.

Tabela 1

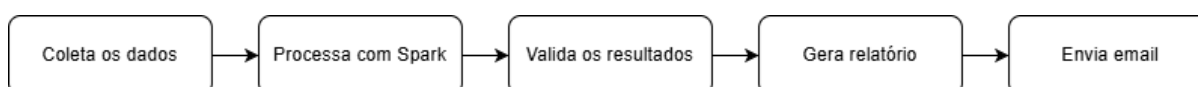
	O que é	Para que serve
DAG	um grafo acíclico que representa o pipeline	Organizar o pipeline de maneira que cada task seja executada na ordem correta
Operator	Descreve uma task em um DAG	Define o que a task irá fazer
Task	Instância de um operator	Descreve o que tem que ser feito
Task Instance	Execução de uma task em um determinado tempo	Registra qual task foi operada, quando foi e qual o resultado
Workflow	Combinação de todos os elementos	A execução do pipeline

O airflow possui duas formas de ser usado, uma delas é por meio de CLI (Command Line Interface) ou seja, é controlado pelo terminal, e também possui uma interface que facilita o uso desta ferramenta como mostrado na (Figura 1).

Figura 1 - UI do airflow



O DAG no airflow, significa um grafo direcionado finito, nisso o instrutor mostra exemplo de loop proibido onde colocar uma task A que depende de B, quando o airflow rodar A vai rodar B e depois A de novo assim fazendo um loop infinito. A responsabilidade de um DAG é garantir a dependência entre tasks e que elas sejam feitas na hora certa e na ordem certa. Na (figura 2) mostra um exemplo de DAG válido.

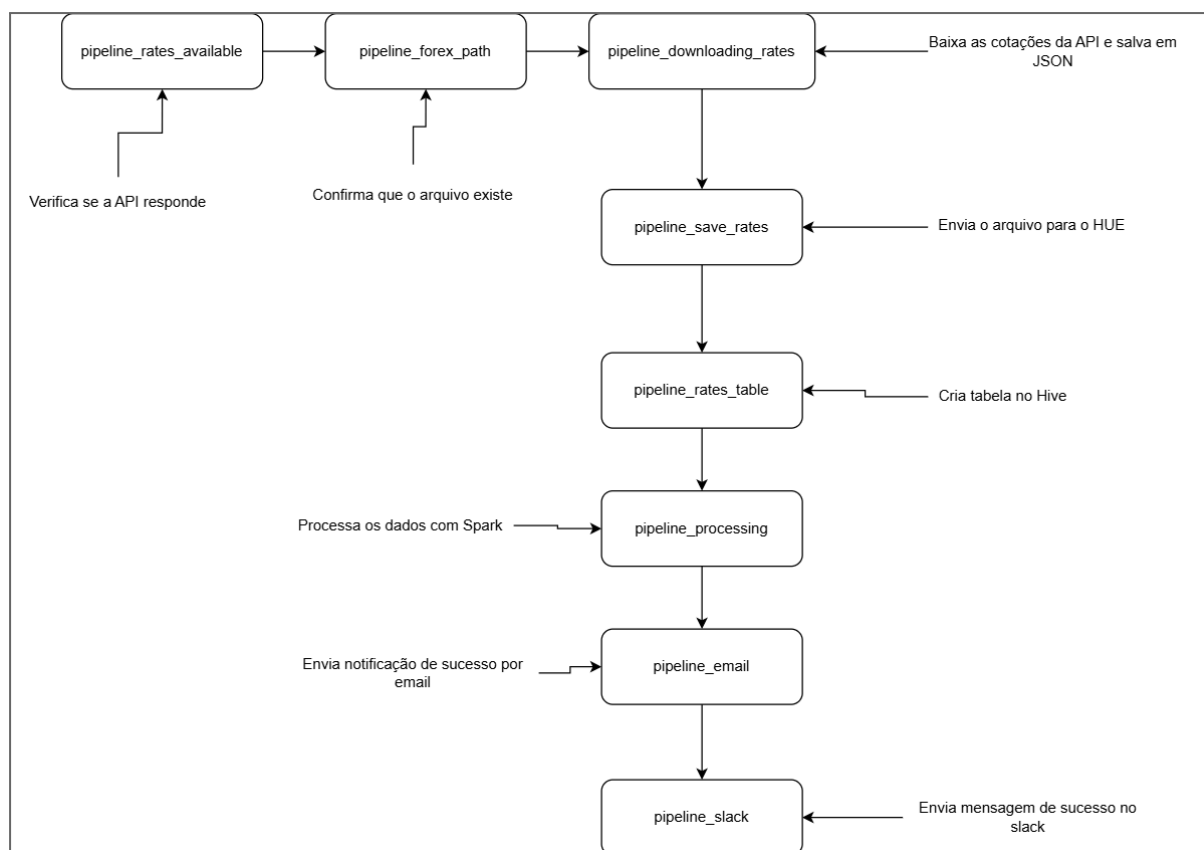


Um Operator no airflow, controla o que será feito pela task. Ele descreve uma task dentro de um pipeline, podendo funcionar sozinho, um exemplo de operator é um script bash que roda um arquivo python para fazer web scraping, ou seja, ele serve como uma forma independente de um recurso da task a ser executada, não sendo necessário acionar o DAG inteiro. Existem três tipos de Operators:

- Action Operators: executam alguma coisa, seja com BashOperator ou PythonOperator;
- Transfer Operators: movem dados para outro sistema, um exemplo o MySql;
- Sensor Operators: esperam que algo aconteça, exemplo é o FileSensor que espero um arquivo chegar em um determinado lugar.

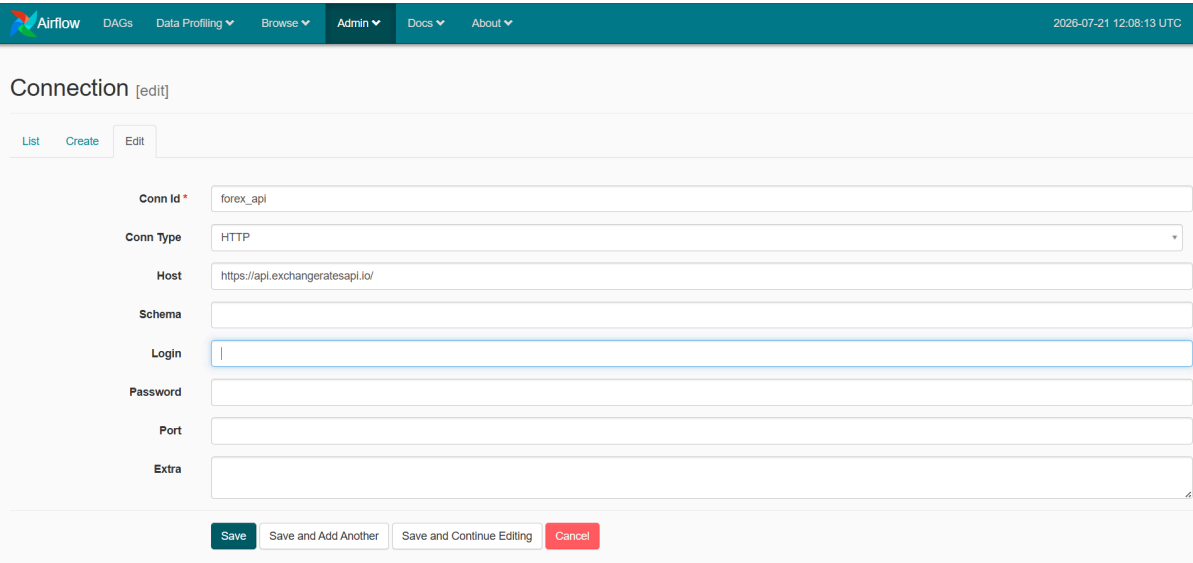
O projeto desenvolvido na seção “The Forex Data Pipeline”, foi um projeto complexo e de muito aprendizado. Algumas mudanças eu tive que fazer em relação aos Dockerfiles e o compose, como já tenho experiência com Docker foi rápido de resolver, os problemas foram que algumas imagens não estavam em versões atualizadas ou foram descontinuadas, porém com as alterações consegui resolver essa questão. Em relação a api usada, as regras dela mudaram desde a gravação dos vídeos, como por exemplo a necessidade de um token para a api, outro problema foi que a api no plano gratuito só retorna a base EUR, não todas as bases como o instrutor mostra. Porém, todas as aulas foram seguidas normalmente e não tive mais problemas de incompatibilidade em relação ao vídeo. A seguir na (Figura 3) é mostrado o pipeline utilizado no airflow.

Figura 3 - Pipeline de execução airflow



Nesse contexto, como pedido pelo card, em seguida será mostrado todos os prints de execução realizados neste projeto (Figura 4), (Figura 5), (Figura 6), (Figura 7), (Figura 8), (Figura 9), (Figura 10), (Figura 11).

Figura 4 - Conexão airflow com api



The screenshot shows the Airflow web interface with the "Connection" form open. The form fields are as follows:

- Conn Id ***: forex_api
- Conn Type**: HTTP
- Host**: https://api.exchangeratesapi.io/
- Schema**: (empty)
- Login**: (empty)
- Password**: (empty)
- Port**: (empty)
- Extra**: (empty)

At the bottom of the form, there are four buttons: "Save", "Save and Add Another", "Save and Continue Editing", and "Cancel".

Figura 5 - Conexão com a pasta do arquivo

Airflow DAGs Data Profiling Browse Admin Docs About 2026-07-21 12:12:32 UTC

Connection [create]

List Create

Conn Id * forex_path

Conn Type File (path)

Host

Schema

Login

Password

Port

Extra {"path": "usr/local/airflow/dags/files"}

Save Save and Add Another Save and Continue Editing Cancel

Figura 6 - Dados em Json

HUE Query Search saved documents... Jobs root

File Browser

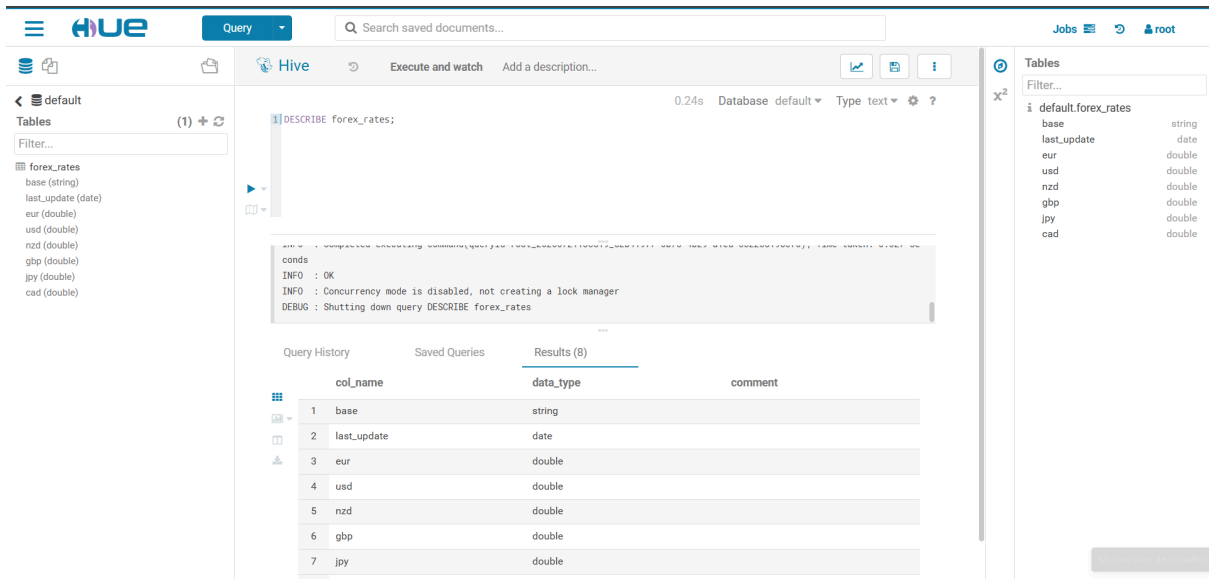
Back Edit file Refresh View as binary Download

Inicio / forex / forex_rates.json Page 1 to 1 of 1

```
{
  "base": "EUR",
  "rates": {
    "USD": 1.141735,
    "NZD": 1.954708,
    "JPY": 185.047607,
    "GBP": 0.852676,
    "CAD": 1.606906,
    "last_update": "2026-07-21"
  }
},
{
  "base": "EUR",
  "rates": {
    "USD": 1.141735,
    "NZD": 1.954708,
    "JPY": 185.047607,
    "GBP": 0.852676,
    "CAD": 1.606906,
    "last_update": "2026-07-21"
  }
}
```

Last modified: 21/07/2026 12:54
Utilizador: airflow
Group: supergroup
Size: 286 B
Mode: 100644

Figura 7 - Criação da tabela

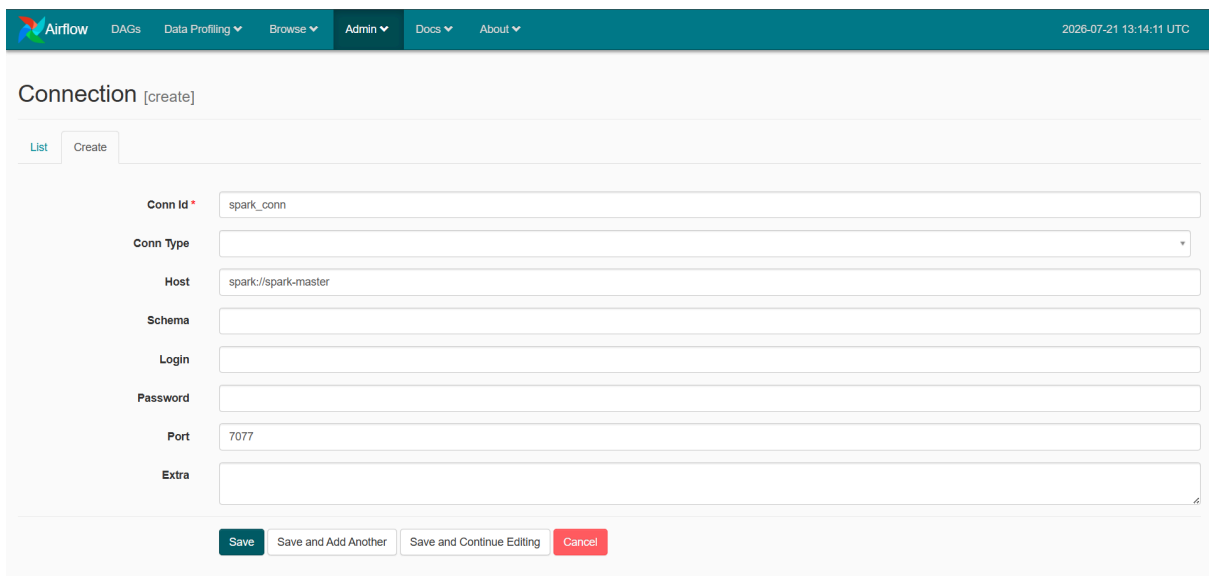


Query: DESCRIBE forex_rates;

Results (8):

col_name	data_type	comment
1 base	string	
2 last_update	date	
3 eur	double	
4 usd	double	
5 nzd	double	
6 gbp	double	
7 jpy	double	

Figura 8 - Conexão airflow com spark



Connection [create]

Conn Id: spark_conn

Conn Type: [dropdown]

Host: spark://spark-master

Schema: [text]

Login: [text]

Password: [text]

Port: 7077

Extra: [text]

Buttons: Save, Save and Add Another, Save and Continue Editing, Cancel

Figura 9 - Inserção na tabela

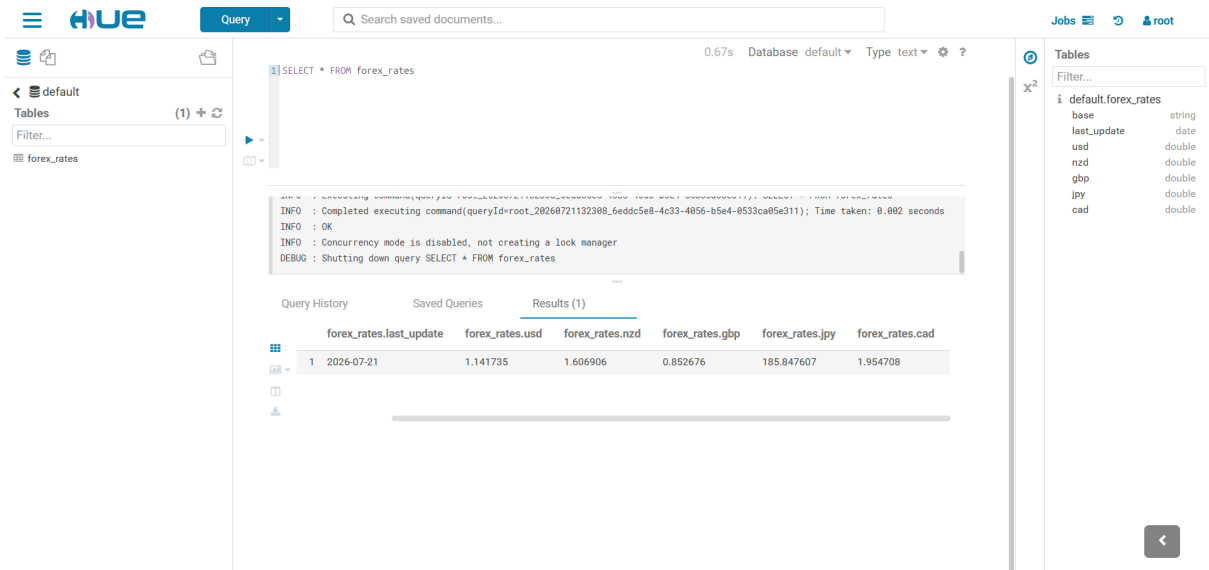


Figura 10 - Pipeline de execução no airflow

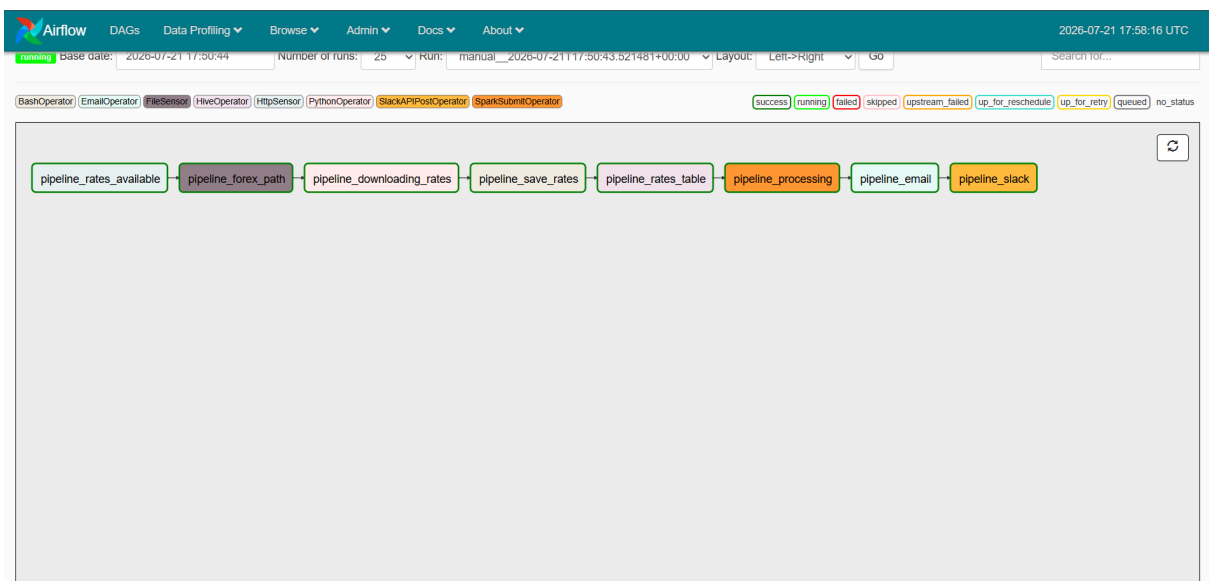
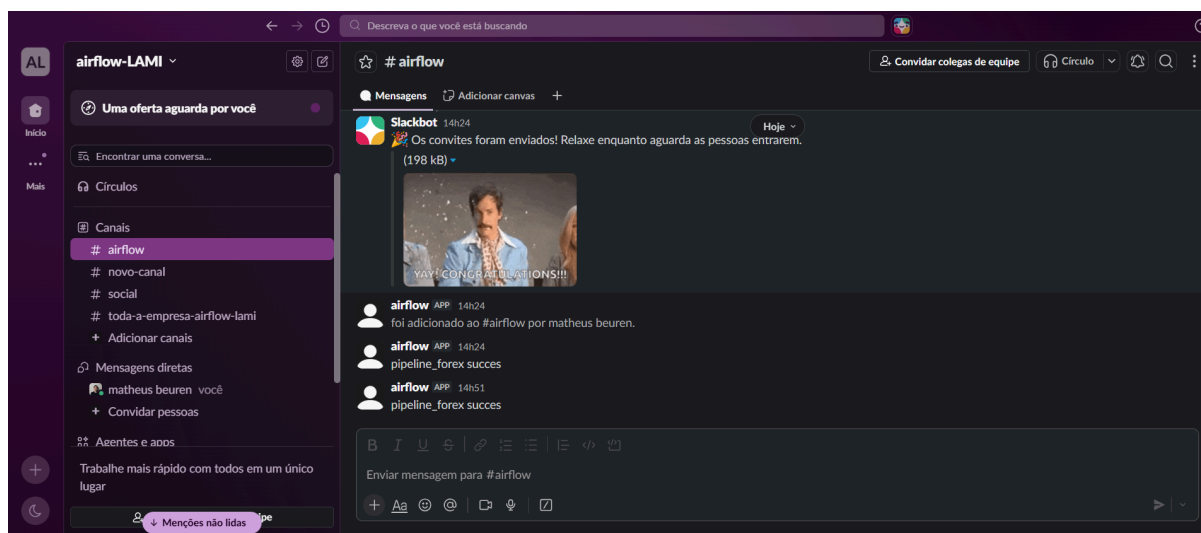


Figura 11 - Mensagem no Slack



Um ponto de muita atenção foi na introdução do 'start_date' e 'scheduler_interval', onde no começo tive dificuldade de entender. O start_date é a data que o scheduler começa a olhar para o dag, e scheduler_interval significa de quanto em quanto tempo executar o dag, a nomenclatura utilizada pelo airflow é "0 * * * *" onde o * seleciona todo tempo disponível, no exemplo que mostrei seria a cada minuto 0 de cada hora de todos os dias. Supondo que um start_date começa às 9:00 e tem um scheduler de 30 minutos, então a execução do dag só será feita às 9:30, pois o scheduler espera o tempo de execução para processar o dag, fazendo assim o execution_date será o start_date, nesse caso 9:00.

Em vista da execução dos dags, se em um determinado momento pausar seu dag e nisso quando volta existem datas que não foram executadas pelo dag, então com o parâmetro default do airflow "catchup=True" todas as execuções atrasadas serão acionadas, caso esteja como "False" não será disparado as execuções atrasadas.

Dois conceitos importantes, o "depends_on_past" verifica se a task com este parâmetro foi executada corretamente, ou seja sem falhas e o "wait_for_downstream" verifica se as tasks (posteriores a ela) da execução anterior foram executadas com sucesso. Com esses parâmetros, diminui o conflito entre execuções de um mesmo dag, em um cenário que existe mais de um dag que manipula o mesmo dado, isso faz com que um não atrole o outro por exemplo, processar um dataset antes que a execução anterior tenha terminado de utilizá-lo.⁷

Em um cenário de falhas, o airflow disponibiliza parâmetros em dois níveis, no dag e em task. No nível dos dags, o 'dagrun_timeout' define um limite de tempo em que um dag é executado, 'on_failure_callback' e 'on_success_callback' são funções chamadas em caso de falha ou sucesso do dag. O mesmo tipo funções de

callback de dags funcionam para as tasks, nisso as tasks também possuem uma maneira de envio de email em caso de falhas e de tentativas.

3. Conclusão

O Airflow resolve o problema do agendamento de execução de pipelines de dados, ou qualquer tipo de pipeline que precisa ser executado manualmente. Ele é útil por conta de ser uma ferramenta que automatiza os processos que por sua vez são manuais na maioria dos casos, nisso o airflow executa por meio de tasks que são definidas pelo próprio dev, gerenciando suas dependências e ordens de execução.